

Day 19 (01/12/2025)

Nested lists - A nested list is a list inside another list.

It means one list can contain multiple lists as its elements.

ex - Seating = $\left[\begin{matrix} 0 & 1 & 2 \\ ["Ravi", "Kiran", "Anu"], & ["Megha", "Sita", "Rahul"], & ["Arjun"] \end{matrix} \right]$

Seating[0] $\Rightarrow$ ['Ravi', 'Kiran', 'Anu']

Seating[1][2] $\Rightarrow$ 'Rahul'

Seating[2][0] $\Rightarrow$ 'Arjun'

$\rightarrow$ extracting each element

```
for i in range(len(seating)):
    for j in range(len(seating[i])):
        print(seating[i][j])
```

o/p $\Rightarrow$

Ravi
Kiran
Anu
Megha
Sita
Rahul
Arjun

Q) Why do we need two loops for nested lists?

Here we have:

A main list (outer list)

Inside it, inner lists (each row)

First loop - goes through each inner list.

Second loop - goes through each item inside that inner list.

Loop 1 (i loop) -

gets each inner list -

* ["Ravi", "Kiran", "Anu"]

* ["Megha", "Sita", "Rahul"]

Loop 2 (j loop)

gets each element inside that inner list:

* "Ravi", "Kiran", "Anu"

* "Megha", "Sita", "Rahul"

* "Arjun"

Q) Add new item to list after a specified item.

write a program to add item 7000 after 6000 in the following python list.

```
list1 = [10, 20, [300, 400, [5000, 6000], 500], 30, 40]
```

expected o/p →

```
[10, 20, [300, 400, [5000, 6000, 7000], 500], 30, 40]
```

```
list1 = [10, 20, [300, 400, [5000, 6000], 500], 30, 40]
```

```
list1[2][2].append(7000)
```

list1

```
[10, 20, [300, 400, [5000, 6000, 7000], 500], 30, 40]
```

list Comprehension -

A concise way to create a new list by processing each element of an iterable.

[expression for item in iterable if condition]

```
# 1, 2, 3, 4, 5
```

```
Squares = [i*i for i in range(1, 6, 1)]
```

Squares

```
[1, 4, 9, 16, 25]
```

we can create only single list.

multiple list is not possible in list comprehension.

Even list.

```
even = [i for i in range(1, 11, 1) if i%2 == 0]
```

even

```
[2, 4, 6, 8, 10]
```


odd list -

```
odd = [i for i in range(1,11,1) if i%2 != 0]
```

odd

[1,3,5,7,9]

tuple: its also multivalued variable which stores heterogenous data and ordered, immutable, allow duplicates.

tuple declared by: () or tuple()

```
t1 = ("poojyy", 45, True, 5+2j, 12.76, 45)
```

t1

('poojyy', 45, True, (5+2j), 12.76, 45)

```
t1[0]
```

'poojyy'

```
→ for i in t1:  
    print(i)
```

poojyy
45
True
(5+2j)
12.76
45

```
→ t1.append("hello")
```

Attribute error - 'tuple' object has no attribute 'append'

```
→ t1.insert(2, "python")
```

Attribute error - 'tuple' object has no attribute 'insert'

```
→ t1[0] = "Lilly"
```

Type Error - 'tuple' object does not support item assignment.

→ `t1.pop()`

AttributeError: 'tuple' object has no attribute 'pop'

→ `t1.remove()`

AttributeError: 'tuple' object has no attribute 'remove'

These errors are happening because tuple is a immutable.

Suppose if we want to do anything like add then we have to convert that tuple in to list & do the changes and again reconvert it in to tuple.

→ `l1 = list(t1)`

`l1`
`['poojya', 45, TrueFalse, (5+2j), 12.76, 45]`

→ `l1[2] = False`

`l1`
`['poojya', 45, False, (5+2j), 12.76, 45]`

→ `t1 = tuple(l1)`

`t1`
`('poojya', 45, False, (5+2j), 12.76, 45)`

here we updating the list which is converted from tuple to list.

Sets: It is also multivalued heterogenous data type and it is unordered, unindexed, mutable and contain only unique items (no duplicates)

declared by: `{ }` or `set()`

→ $S1 = \{ "Java", 12, 156.57, True, False, True, 1 \}$

$S1$

$\{ 12, 156.57, False, True, 'Java' \} \rightarrow$ here it is unordered

① $\downarrow$ int ② $\downarrow$ float ③ $\swarrow$ Boolean ④ $\downarrow$ string

→ if we try to extract $S1[0]$

Type Error - 'Set' object is not subscriptable.

→ for i in $S1$:
 $\text{print}(i)$

False

True

Java

156.57

12

Operations in Set -

1) Union (|) → Combines ~~all~~ elements from both sets without duplicates.

$S1 = \{ 1, 2, 3, 4, 5 \}$

$S2 = \{ 4, 5, 6, 7, 8 \}$

$S1.union(S2)$

(or)

$S1 = \{ 1, 2, 3, 4, 5 \}$

$S2 = \{ 4, 5, 6, 7, 8 \}$

$\text{print}(S1 | S2)$

o/p $\{ 1, 2, 3, 4, 5, 6, 7, 8 \}$

o/p $\{ 1, 2, 3, 4, 5, 6, 7, 8 \}$

2) Intersection (&) → gives common elements from both sides.

$S1 = \{ 1, 2, 3, 4, 5 \}$

$S2 = \{ 4, 5, 6, 7, 8 \}$

$S1.intersection(S2)$

o/p $\{ 4, 5 \}$

$S1 = \{ 1, 2, 3, 4, 5 \}$

$S2 = \{ 4, 5, 6, 7, 8 \}$

$\text{print}(S1 \& S2)$

o/p $\{ 4, 5 \}$

3) Difference(-) gives elements present in one set but not present in the other.

$$S_1 = \{1, 2, 3, 4, 5\}$$

$$S_2 = \{4, 5, 6, 7, 8\}$$

S_1 . difference(S_2)

o/p $\{1, 2, 3\}$

S_2 . difference(S_1)

o/p $\{6, 7, 8\}$

$$S_1 = \{1, 2, 3, 4, 5\}$$

$$S_2 = \{4, 5, 6, 7, 8\}$$

print($S_1 - S_2$)

o/p $\{1, 2, 3\}$

print($S_2 - S_1$)

o/p $\{6, 7, 8\}$

4) Symmetric Difference (^) gives elements that are in either of the sets but not in both.

$$S_1 = \{1, 2, 3, 4, 5\}$$

$$S_2 = \{4, 5, 6, 7, 8\}$$

S_1 . Symmetric-difference(S_2)

o/p $\{1, 2, 3, 6, 7, 8\}$

$$S_1 = \{1, 2, 3, 4, 5\}$$

$$S_2 = \{4, 5, 6, 7, 8\}$$

print($S_1 \wedge S_2$)

o/p $\{1, 2, 3, 6, 7, 8\}$

5) Subset and Superset.

Subset - A subset is a set whose all elements are completely contained inside another set. ($\leq$)

$\Rightarrow$ If every element of set A is present in set B, then A is a subset of B.

ex - $A = \{1, 2\}$

$$B = \{1, 2, 3, 4\}$$

print($A \leq B$)

o/p - True $\rightarrow$ because 1 and 2 are present in B, A is a subset of B.

Superset - A Superset is a set that contains all elements of another set. ($\supseteq$)

$\rightarrow$ if Set A contains all elements of set B, A is a superset of B.

* $A = \{1, 2, 3, 4\}$

$B = \{1, 2\}$

`print(A >= B)`

o/p: True because A contains 1 & 2, A is a superset of B.

Subset - "inside" $\{1, 2\}$ is inside $\{1, 2, 3, 4\}$.

Superset - "contain" $\{1, 2, 3, 4\}$ contains $\{1, 2\}$.

6) Add & Remove elements -

Add -

$S = \{1, 2, 3\}$

`S.add(4)`

`print(S)` $\Rightarrow \# \{1, 2, 3, 4\}$

Remove -

`S.remove(2)`

`print(S)`

$\Rightarrow \{1, 3, 4\}$

`S.pop()`

$\{3, 4\}$

$\hookrightarrow$ performing from beginning.

discard -

`S.discard(10)`

`print(S)`

$\Rightarrow$ No error even if element doesn't exist.

clear -

`S.clear()`

`print(S)`

$\Rightarrow$ set()

$\rightarrow$ for list, tuple, set, dictionary everything same.